

Structured Retrieval for a Growing Lean Artifact-Proof Library

Anonymous manuscript

9 August 2026

Abstract

A coding agent constructing direct Lean proofs about WebAssembly artifacts needs a library of lemmas, tactics, and guidance, but loading a growing library into every prompt consumes context and exposes artifact-specific examples during evaluation. We describe the structured lemmas, tactics, and guidance (LTG) catalog implemented in LeanExe: canonical entries with machine-readable metadata and prose guidance, overlapping generated JSON Lines (JSONL) category indexes, checked declaration references, filters that remove examples for the same or closely related artifacts, frozen task-specific views, and proof journals that record retrieval decisions. Two trials held a specification, source program, WebAssembly artifact identified by the same digest, decoded program, toolchain, and checked Lean modules fixed. A separate package verifier rebuilt the exact-artifact theorem and accepted both proofs. The agents selected relevant entries, rejected unrelated entries from index summaries, and exposed missing aliases and application guidance. A synthetic 10,000-record index query returned its two relevant records with less than 10 KB of output. The trials establish selective and reproducible discovery at the current scale. Both took longer than the median of three prior runs with the same artifact and the same checked theorem for its scalar loop, so the measurements do not establish lower proof-generation time.

1 Artifact-proof libraries need selective discovery

Lean proof automation ranges from tactic-level search to language-model agents that retrieve premises and edit proof scripts [4, 5, 7]. Systems with growing libraries must distinguish reusable abstractions from single-use declarations and must control which library material enters an evaluation task [1, 6]. Direct artifact proofs add two local constraints. Generated WebAssembly contains recurring compiler and runtime instruction patterns whose useful names may come from annotations rather than the current Lean goal, and a worked proof for the same artifact can disclose the answer that an evaluation intends the agent to construct.

LeanExe proves functional properties directly about frozen WebAssembly binaries in Lean 4 [2, 3]. The proof kit comprises the checked Lean modules that an artifact proof may import, including semantic lemmas, tactics, and theorems for recurring instruction regions. We call such a region a compiler or runtime motif when an annotation identifies it as the output of one compiler or runtime construction. LTG adds application guidance and worked examples to the checked proof kit. The reported snapshot had twelve canonical entries in seven categories, while reading every entry would make prompt context grow with the full catalog.

The central design claim is limited: canonical entry bodies, overlapping indexes, task-specific filtering, and retained retrieval records make proof support selectively discoverable and later auditable. Lean elaborates a theorem application into a proof term, and its kernel checks the resulting declaration. Catalog metadata proposes relevant material but grants no import, introduces no premise, and proves no relation between a compiler annotation and decoded artifact instructions.

2 Canonical entries generate overlapping indexes

Each LTG entry has one directory containing an `entry.json` record and a prose `README.md`. The record assigns roles that distinguish checked proof assets, annotation support, guidance, proof-generation methods, and worked examples. It assigns a scope of generic semantics, compiler or runtime motif, or benchmark-local material, as well as an evidence status. A promoted evidence status favors an entry during retrieval, provisional marks support still under evaluation, and rejected records a failed candidate. Promotion changes a provisional entry to promoted after recurring evidence justifies that preference. The record also lists features, annotation kinds,

File	Function
<code>categories.json</code>	Lists category identifiers and their search scopes.
<code>categories/*/tools.jsonl</code>	Filters summaries and exact proof vocabulary with ordinary lexical search.
<code>entries/*/entry.json</code>	Records canonical identity, classification, declarations, relations, and exclusions.
<code>entries/*/README.md</code>	Explains application order, proof obligations, and limitations.
<code>LTGCheck.lean</code>	Resolves every advertised declaration in the checked proof workspace.
<code>LTG_TASK.json</code>	Identifies the filtered task view and its content digest.

Table 1: Files in the structured LTG retrieval interface.

Lean modules, declarations, premises, result, demos whose accepted proofs use it, related entries, and exclusion policy.

Entries appear in one or more categories such as arrays, loops, memory, allocation, compiler motifs, proof construction, and worked examples. A generated `tools.jsonl` file in each category contains one record per applicable entry. Each record includes the title, summary, features, annotation kinds, exact module and declaration names, derived search terms, and the relative path to the canonical entry. The reported root category file is 1.3 KB, and the seven category indexes contain 46 references to the twelve overlapping entries in 51 KB. The agent can therefore search a generated annotation kind, a residual theorem name, or an application term before reading any entry body.

The generator derives every category index from the canonical records. It also generates a Lean module that imports all advertised proof-kit modules and issues `#check` commands for every advertised declaration. Repository checks reject malformed records, unknown category or related-entry references, proof modules outside the allowed proof kit, and stale generated files. A retrieval failure in the first trial exposed missing declaration aliases, and a later test requires the revised index to answer that first query.

3 Task views prevent direct example disclosure

The proof orchestrator constructs a filtered LTG view after fixing the artifact. An entry may exclude one or more exact artifact digests and name a derivative group, an identifier for artifacts close enough that a worked proof may disclose the evaluated proof. The filter removes the canonical body and every category-index reference, so another category cannot disclose an excluded worked example. Exact-artifact filtering runs automatically, while evaluation code must currently supply derivative groups because the orchestrator does not classify artifact relationships.

The task manifest records the artifact digest, supplied derivative groups, sorted included and excluded entry identifiers, counts, and a digest over every visible LTG file. Version 7 of the proof-package format archives the same filtered directory and manifest that the agent received. Package validation recomputes the digest, checks category references, requires every included entry to have an indexed metadata record and prose body, and rejects any archived body for an excluded entry. Later analysis can reconstruct the available proof support without consulting the current repository catalog.

The prompt gives the agent a retrieval protocol and access to the filtered file tree. The complete unfiltered view at the reported snapshot contains 33 files and 81 KB, while the prompt directs the agent first to the 1.3 KB category list. The agent searches likely JSONL indexes with ordinary file tools, opens entry bodies selected by the summaries, and follows related-entry links when the proof state supports them. Its prose journal records exact queries, entries opened, entries used, entries rejected, and missing support. The journal lets a later catalog revision address failed vocabulary and irrelevant matches, while the accepted Lean source shows which shared declarations the proof references.

4 Fixed-artifact trials establish selective discovery

We tested the retrieval protocol on a fixed Demo 6 task whose public function maps a singleton `Array UInt64` containing x to a singleton containing $\text{gcd}(x, 42)$ and preserves arrays of every other length. Both structured trials held the formal specification, generated source, WebAssembly binary digest, decoded program, compiler

annotations, toolchain, and checked proof kit fixed. The package verifier independently rebuilt and accepted both resulting exact-artifact proofs.

The first agent searched the array, compiler-motif, loop, allocation, and proof-construction indexes. It used the scalar post-test loop, singleton-wrapper, and residual-normalization entries and rejected counter-transfer examples from their summaries. The journal exposed two catalog defects: its first wrapper query found no entry because declaration aliases were missing from index search terms, and no entry described the Euclidean invariant. The package verifier accepted the proof, and the proof stage took 881 seconds.

The revised catalog added module and declaration aliases and a provisional Euclidean-GCD guidance entry. The second agent searched four category indexes, opened the wrapper, residual-normalization, scalar-loop, and Euclidean entries, and used all four. It rejected counter, map, and filter entries from their summaries without opening their bodies. The package verifier accepted the proof, and the proof stage took 650 seconds.

The comparison series consists of three earlier runs that held the same formal specification, source, binary digest, decoded program, annotations, toolchain, and checked theorem for the scalar loop fixed but did not use the structured retrieval instructions or LTG tree. Those runs took 495, 520, and 511 seconds, for a median of 511 seconds and a range of 25 seconds. The structured runs took 881 and 650 seconds. Agent runtime varies, the real catalog contains twelve entries, and the experiment includes one artifact. These data support claims about selective discovery and accepted proof construction, while they provide negative evidence for a proof-time improvement in this case.

A separate file-level test placed the real scalar-loop and Euclidean entries among 9,998 synthetic records in one JSONL index. The Demo 6 query returned those two records and no others, producing less than 10 KB of output. The test establishes lexical selectivity for one query but does not measure agent judgment.

5 Evidence governs catalog revision

The catalog distinguishes preservation from promotion. A narrow worked example can remain searchable because it records useful proof organization, while exact-artifact and derivative filters keep it out of the tasks it answers. Use of a checked declaration by multiple demos can justify favoring an entry during retrieval or promoting it. Specificity alone does not require deletion, and one successful consumer does not establish a generic abstraction.

Each proof iteration produces three forms of evidence: the accepted Lean source, the agent journal, and task telemetry. The source identifies artifact-local definitions, repeated proof terms, and shared theorem use, while the journal identifies failed retrieval and missing explanations. Telemetry records proof time, revisions, and other process measurements. Reviewing the three records together led directly to declaration aliases and Euclidean guidance after the first structured trial. A time reduction is one possible result rather than a retention condition.

The real catalog contains twelve entries, and its fixed-artifact retrieval evidence comes from one program. Search is lexical, derivative-group filtering requires external classification, and the experiments do not measure prompt tokens or time to the first useful theorem application. Further evaluation needs diverse held-out artifacts and larger real catalogs, with retained records of queries, files opened, context consumed, accepted theorem use, revisions, proof structure, and elapsed time. Those measurements can distinguish index selectivity from the harder question of whether an agent chooses useful proof support.

6 Conclusion

Structured LTG turns a proof library into a checked, searchable, and task-specific file interface. Canonical entry bodies prevent divergent copies, generated JSONL indexes provide overlapping lexical views, and frozen filtered views make later evaluation reproducible. Journals connect retrieval failures to catalog revisions. Lean’s kernel checks the accepted proof declaration, while the package verifier checks the archived LTG identity and rebuilds the exact-artifact theorem.

The reported implementation demonstrates selective retrieval from a twelve-entry real catalog and a 10,000-record synthetic index. Two agents in the fixed-artifact trials found and used relevant support, and the package verifier accepted both proofs. Their elapsed times do not show a speed improvement, and broader held-out evidence remains necessary before drawing conclusions about proof-generation efficiency.

References

- [1] Ian Berlot-Attwell, Frank Rudzicz, and Xujie Si. Library learning doesn’t: The curious case of the single-use “library”. *arXiv preprint arXiv:2410.20274*, 2024. doi: 10.48550/arXiv.2410.20274.
- [2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.
- [3] LeanExe contributors. LeanExe: Compilation and exact-artifact verification for Lean and WebAssembly. Software repository, 2026. URL <https://github.com/jsmorph/leanexe>. Structured LTG snapshot 0745b82.
- [4] Jannis Limperg and Asta Halkjær From. Aesop: White-box best-first proof search for Lean. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 253–266. Association for Computing Machinery, 2023. doi: 10.1145/3573105.3575671.
- [5] Amitayush Thakur, George Tsoukalas, Yeming Wen, Jimmy Xin, and Swarat Chaudhuri. An in-context learning agent for formal theorem-proving. In *Proceedings of the First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=V7HRrxXUhn>.
- [6] Haiming Wang, Huajian Xin, Chuanyang Zheng, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, and Xiaodan Liang. LEGO-Prover: Neural theorem proving with growing libraries. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3f5PALef5B>.
- [7] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.52202/075280-0944.